OFFSHORE-TO-NEARSHORE WAVE TRANSFORMATION

Abstract

This repository implements a deterministic offshore-to-nearshore wave-transformation model for time-series forcing data stored in CSV format. The program reads offshore significant wave height, mean wave period, and mean wave direction, propagates each valid sea state to a prescribed local depth, and writes both a transformed time series and a statistical report.

The implemented workflow combines linear wave dispersion, directional screening relative to a user-supplied coastline orientation, refraction by a Snell-type kinematic transformation, shoaling through linear group-velocity relations, and a Miche-type breaking limitation.

The code is designed as a practical engineering post-processing tool rather than as a spectral wave model. It does not solve two-dimensional phase-resolving hydrodynamics, diffraction, current-wave interaction, spectral spreading, or energy dissipation over complex bathymetry. Instead, it applies a compact and computationally efficient one-point transformation to each record independently.

This README documents the implemented behaviour of `transpose.cpp`, including input requirements, numerical formulation, directional conventions, filtering rules, output files, report logic, and compilation.

---

1. Scope and engineering purpose

The program transforms offshore wave conditions to a nearshore reference point at constant depth `depth_d` using a single-record, non-spectral formulation. For each valid time step, the code computes:

- deep-water wavelength from the input mean wave period;
- finite-depth wavelength from the linear dispersion relation;
- dimensionless depth `kh`;
- signed offshore obliquity relative to the coastline;
- refracted local obliquity;
- transformed local mean wave direction;
- shoaling coefficient;
- refraction coefficient;
- Miche-type breaking height;
- final local significant wave height capped by breaking.

The tool is intended for rapid, auditable transformation of long offshore time series to a fixed nearshore depth. It is suitable for screening studies, sensitivity testing, preprocessing of offshore forcing, and preparation of simplified nearshore design series where a transparent one-point transformation is acceptable.

---

2. Input data model

The executable expects a comma-separated file whose header starts exactly with the following four columns:

```
datetime,swh,mwp,mwd
```

The program reads only these first four columns. Additional columns may be present after `mwd`, but they are ignored by the calculation and by the report.

2.1 Required input variables

- `datetime`: timestamp string. The code keeps it as text and extracts the year from its first four characters for annual maxima.
- `swh`: offshore significant wave height.
- `mwp`: offshore mean wave period. This is the period used in all wavelength, dispersion, shoaling, refraction, and breaking calculations.
- `mwd`: offshore mean wave direction.

2.2 Required column order

The required fields must be the first four CSV columns and must appear in this order:

```
datetime,swh,mwp,mwd
```

The header validation is case-insensitive and trims surrounding blanks or quotation marks. However, the first four field names must still correspond to `datetime`, `swh`, `mwp`, and `mwd` after trimming and lower-casing.

A valid extended input header is therefore, for example:

```
datetime,swh,mwp,mwd,wind,dwi,u10,v10
```

Only `datetime`, `swh`, `mwp`, and `mwd` are used.

2.3 Implied units and conventions

The implementation assumes:

- `swh` in metres;
- `mwp` in seconds;
- `mwd` in degrees clockwise from North;
- `coast_dir` in degrees clockwise from North;
- `depth_d` in metres.

The program does not perform unit conversion. Inputs must already be expressed in a consistent unit system.

2.4 Direction convention

The code uses the standard metocean convention in which mean wave direction is the direction **from which waves come**, expressed clockwise from geographic North.

If the input dataset uses a going-to convention or a different angular reference system, the directions must be converted before running the executable.

---

3. Program outputs

Running the executable produces two files in the working directory:

- `output.csv`;
- `report.txt`.

3.1 `output.csv`

The output CSV contains one transformed row per retained timestamp with the following columns:

```
datetime,swh_offshore,mwd_offshore,mwp,L0,L,kh,alpha_offshore,alpha_local,swh_local,mwd_local,Ks,Kr,
```

The numeric output is written with fixed decimal notation and ten decimal places.

3.2 `report.txt`

The report contains:

- the exact command line used to run the executable;
- descriptive statistics for all reported variables;
- a note explaining the filtering applied to selected local variables;
- annual maxima of `swh_offshore` and `swh_local`;
- overall maxima across all reported years.

---

4. Meaning of command-line arguments

The executable is called as follows:

```
./transpose input_csv coast_dir depth_d
```

where:

- `input_csv`: input CSV file whose first four columns are `datetime,swh,mwp,mwd`;
- `coast_dir`: coastline azimuth in degrees clockwise from North;
- `depth_d`: target local depth in metres.

Example:

```
./transpose input.csv 270 8.0
```

This reads `input.csv`, uses a coastline azimuth of `270°`, and transforms the offshore records to a local depth of `8.0 m`.

The program requires exactly three arguments after the executable name. It stops with an error if `coast_dir` or `depth_d` cannot be parsed as numbers, or if `depth_d <= 0`.

---

5. Meaning of `coast_dir`

`coast_dir` is the coastline azimuth, not the seaward normal. It represents the orientation of the shoreline itself, measured clockwise from North.

Examples:

- `coast_dir = 0` or `180` means a coastline aligned North-South;
- `coast_dir = 90` or `270` means a coastline aligned East-West.

The code normalizes `coast_dir` to `[0, 360)` before using it.

This distinction is important because the code computes wave obliquity relative to the coastline line, then reconstructs the local mean wave direction after refraction using that signed obliquity.

---

6. Physical and mathematical formulation

6.1 Governing assumptions

The implementation follows these assumptions:

1. Each time step is treated independently.
2. Linear wave theory is used for wavelength, celerity, group velocity, shoaling, and refraction.
3. Refraction is represented through a local Snell-type relation based on the ratio `C/C0`.
4. Breaking is imposed through a Miche-type upper bound.
5. Waves identified as arriving from the land side are suppressed and assigned zero local height.
6. No diffraction, bottom friction, wind input, currents, nonlinear interactions, spatial ray tracing, or spectral spreading are modelled.

These assumptions define the tool as a compact point-transformation model.

6.2 Deep-water wavelength

For a wave period `T = mwp`, the deep-water wavelength is computed as:

$$L_0 = \frac{gT^2}{2\pi},$$

where:

- `g = 9.80665 m/s²`;
- `T` is the input mean wave period.

If `T <= 0`, the code returns `L0 = 0`.

6.3 Local wavelength from the linear dispersion relation

At target depth `d = depth_d`, the finite-depth wavelength `L` is obtained from the implicit linear dispersion equation:

$$L = L_0 \tanh\left(\frac{2\pi d}{L}\right).$$

This is equivalent to:

$$\omega^2 = gk \tanh(kd),$$

with:

$$L = \frac{2\pi}{k}, \qquad \omega = \frac{2\pi}{T}.$$

6.3.1 Initial estimate The code first computes:

$$k_0 d = \frac{2\pi d}{L_0}.$$

It then forms the initial estimate:

$$(kh)_{\text{approx}} = \frac{k_0 d}{\tanh\left[\left(\frac{6}{5}\right)^{k_0 d}\sqrt{k_0 d}\right]}.$$

From this value, the initial wavelength is:

$$L_{\text{init}} = \frac{2\pi d}{(kh)_{\text{approx}}}.$$

If the estimate is unusable, the code uses an Eckart-type fallback:

$$L_{\text{Eckart}} = L_0\sqrt{\tanh\left[(k_0 d)^{1.25}\right]}.$$

The initial wavelength is capped so that `L <= L0`. If it remains unusable, the code falls back to:

$$L = L_0 \tanh\left(\sqrt{k_0 d}\right),$$

and finally to `L = L0` if needed.

6.3.2 Newton-Raphson iteration After initialization, the code solves:

$$F(L) = L - L_0 \tanh\left(\frac{2\pi d}{L}\right) = 0$$

with Newton-Raphson iteration:

$$L_{n+1} = L_n - \frac{F(L_n)}{F'(L_n)}.$$

The derivative used is:

$$F'(L) = 1 + L_0 \frac{2\pi d}{L^2}\left[1 - \tanh^2\left(\frac{2\pi d}{L}\right)\right].$$

The hard-coded iteration parameters are:

- maximum iterations: `20`;
- convergence tolerance: `1e-12`.

If a Newton update produces `L <= 0`, the iteration stops and the current estimate is retained.

6.4 Wavenumber and dimensionless depth

Once `L` is known, the local wavenumber is:

$$k = \frac{2\pi}{L},$$

and the local dimensionless depth is:

$$kh = kd.$$

If `L` is not positive, the code sets `k = 0` and `kh = 0`.

6.5 Directional screening relative to the coastline

The offshore mean wave direction is first normalized to `[0, 360)`.

The code then computes the relative wave direction with respect to the coastline azimuth:

$$\text{relativeDir} = (\text{mwd}_{\text{offshore}} - \text{coast\_dir}) \bmod 360.$$

A wave is treated as arriving from the land side when:

$$0 < \text{relativeDir} < 180.$$

For those records, the code writes zero for all locally transformed quantities. This is a hard screening rule, not a gradual attenuation.

The cases `relativeDir = 0` and `relativeDir >= 180` are treated as sea-side cases by the implementation.

6.6 Signed offshore obliquity `alpha_offshore`

The code computes a signed crest-to-coast angle called `alpha_offshore`.

First, the relative direction is computed as above. The wave crest orientation is then obtained from the offshore mean wave direction:

- if `relativeDir < 180`, the crest angle is `mwd - 90°`;
- otherwise, the crest angle is `mwd + 90°`.

The crest angle is wrapped to `[0, 360)`. The signed difference between crest direction and coastline azimuth is then reduced to `[-180, 180)`.

This sign is preserved during the refraction calculation.

6.7 Refraction and local obliquity `alpha_local`

The code applies a Snell-type transformation using the linear-wave celerity ratio:

$$\frac{C}{C_0} = \frac{L/T}{L_0/T} = \frac{L}{L_0} = \tanh(kh).$$

The local obliquity is therefore computed from:

$$\sin(\alpha_{\text{local}}) = \sin(\alpha_{\text{offshore}}) \tanh(kh),$$

so that:

$$\alpha_{\text{local}} = \arcsin\left[\sin(\alpha_{\text{offshore}}) \tanh(kh)\right].$$

The sine argument is clamped to `[-1, 1]` before evaluating `asin` to avoid floating-point domain errors.

6.8 Local mean wave direction `mwd_local`

After refraction, the local mean wave direction is reconstructed as:

$$\text{mwd}_{\text{local}} = \text{mwd}_{\text{offshore}} - (\alpha_{\text{offshore}} - \alpha_{\text{local}}).$$

The result is wrapped to `[0, 360)`.

This reconstruction follows the sign convention used internally by the code.

6.9 Shoaling coefficient `Ks`

The shoaling coefficient is based on linear energy-flux conservation:

$$K_s = \sqrt{\frac{C_{g0}}{C_g}}.$$

Using:

$$n = \frac{1}{2}\left(1 + \frac{2kh}{\sinh(2kh)}\right),$$

and:

$$\frac{C}{C_0} = \tanh(kh),$$

the implemented expression is:

$$K_s = \sqrt{\frac{1}{\tanh(kh)\,2n}}.$$

Equivalently:

$$K_s = \left[\tanh(kh)\left(1 + \frac{2kh}{\sinh(2kh)}\right)\right]^{-1/2}.$$

The code returns `Ks = 1.0` if the inputs are non-physical or if a denominator becomes too small or non-positive.

6.10 Refraction coefficient `Kr`

The refraction coefficient is computed as:

$$K_r = \sqrt{\frac{\cos(\alpha_{\text{offshore}})}{\cos(\alpha_{\text{local}})}}.$$

The code evaluates this expression only if:

- `cos(alpha_offshore) >= 0`;
- `cos(alpha_local) > tolerance`.

Otherwise, it uses the fallback value:

$$K_r = 1.$$

This avoids unstable amplification near grazing incidence or invalid square-root conditions.

6.11 Miche-type breaking height `Hb`

The breaking-limited height is computed as:

$$H_b = 0.142L\tanh(kh).$$

If `L <= tolerance` or `kh <= 0`, the code sets `Hb = 0`.

6.12 Local significant wave height `swh_local`

The transformed height before breaking limitation is:

$$H_{\text{trans}} = H_{s,\text{offshore}} K_s K_r.$$

The final local significant wave height is:

$$H_{s,\text{local}} = \min(H_{\text{trans}}, H_b).$$

If `Hb <= 0`, the code uses `H_trans` directly. Any negative final value is clipped to zero.

---

7. Algorithmic workflow

7.1 Header validation

The program opens the input CSV and reads the first line as the header. The header must contain at least four columns and must start with:

```
datetime,swh,mwp,mwd
```

If this condition is not met, the program stops with an error.

7.2 Input parsing

After the header, the program:

- reads all remaining non-empty lines;
- ignores purely blank or whitespace-only rows;
- splits each retained row by commas;
- parses the first four fields as `datetime`, `swh`, `mwp`, and `mwd`.

The CSV parser is simple and comma-based. It is appropriate for ordinary numeric CSV files without embedded commas inside quoted fields.

7.3 Temporal ordering and duplicate removal

The program sorts the input records lexicographically by the first field, interpreted as the datetime string.

After sorting, duplicate timestamps are removed by retaining only one line for each distinct datetime string.

This means:

- the output is sorted by datetime string, not by original input order;
- duplicate timestamps are not all preserved;
- duplicate rows with the same timestamp should not be assumed to preserve the original file order after sorting.

7.4 Per-record calculation

For each retained record, the code applies the following logic:

1. Parse `datetime`, `swh`, `mwp`, and `mwd`.
2. If parsing fails, write the original available values and zero for all derived variables.
3. If `swh <= 0` or `mwp <= 0`, write zero for all derived variables.
4. Normalize `mwd` to `[0, 360)`.
5. Compute `relativeDir` from `mwd` and `coast_dir`.
6. If `0 < relativeDir < 180`, classify the record as land-side and write zero for all local transformed variables.
7. Otherwise compute `L0`, `L`, `kh`, `alpha_offshore`, `alpha_local`, `swh_local`, `mwd_local`, `Ks`, `Kr`, and `Hb`.

7.5 Parallel execution

The per-record transformation loop is parallelized with OpenMP:

```
#pragma omp parallel for schedule(static)
```

Thread-local maps are used for annual maxima. These maps are merged after the parallel loop.

Output rows are first stored in memory and then written sequentially, so the final `output.csv` follows the sorted and deduplicated timestamp order.

---

8. Output variables

8.1 `datetime`

Timestamp string copied from the input row.

8.2 `swh_offshore`

Offshore significant wave height read from input column `swh`.

8.3 `mwd_offshore`

Offshore mean wave direction read from input column `mwd`, wrapped to `[0, 360)` for valid parsed records.

8.4 `mwp`

Offshore mean wave period read from input column `mwp`. This is the period variable used by the implemented wave transformation.

8.5 `L0`

Deep-water wavelength computed from `mwp` using linear theory.

8.6 `L`

Finite-depth wavelength computed from `mwp` and `depth_d` by solving the linear dispersion relation.

8.7 `kh`

Dimensionless depth parameter:

$$kh = kd.$$

8.8 `alpha_offshore`

Signed offshore crest-to-coast obliquity in degrees.

8.9 `alpha_local`

Signed local obliquity in degrees after refraction.

8.10 `swh_local`

Final local significant wave height after shoaling, refraction, and breaking limitation.

8.11 `mwd_local`

Local mean wave direction reconstructed from the offshore direction and the change from `alpha_offshore` to `alpha_local`.

8.12 `Ks`

Shoaling coefficient.

8.13 `Kr`

Refraction coefficient.

8.14 `Hb`

Miche-type breaking height.

---

9. Statistical report methodology

The report covers the following thirteen variables:

1. `swh_offshore`
2. `mwd_offshore`
3. `mwp`
4. `L0`
5. `L`
6. `kh`
7. `alpha_offshore`
8. `alpha_local`
9. `swh_local`
10. `mwd_local`
11. `Ks`

12. `Kr`
13. `Hb`

9.1 Linear statistics

For non-directional variables, the code computes:

- count;
- mean;
- sample standard deviation;
- minimum;
- 1st percentile;
- 10th percentile;
- 25th percentile;
- median;
- 75th percentile;
- 90th percentile;
- 99th percentile;
- maximum.

Percentiles are computed after sorting the sample and applying linear interpolation between neighbouring ranks.

9.2 Hybrid circular statistics for directions

For `mwd_offshore` and `mwd_local`, the code uses hybrid directional statistics:

- all directions are wrapped to `[0, 360)`;
- mean is computed as a circular mean using unit vectors;
- standard deviation is computed as circular standard deviation from the mean resultant length;
- minimum, maximum, median, and percentiles are computed linearly on the wrapped angles.

This means that directional means and standard deviations are circular, while directional quantiles are ordinary wrapped-angle quantiles.

9.3 Filtering of local variables

The following variables are filtered before descriptive statistics are computed:

- `alpha_local`;
- `swh_local`;
- `mwd_local`;
- `Ks`;
- `Kr`;
- `Hb`.

For these variables, the code excludes all records for which:

$$swh_{\text{local}} \leq 10^{-12}.$$

This removes land-side waves and records whose final local wave height is effectively zero from the local-variable statistics.

All other variables are summarized over the full retained record set.

9.4 Annual maxima

The report extracts the year from the first four characters of `datetime` and computes annual maxima for:

- `swh_offshore`;
- `swh_local`.

It then prints a final overall maximum row across all reported years.

---

10. Edge cases and safeguards

10.1 Invalid input file or arguments

The program stops with an error if:

- the number of command-line arguments is not correct;
- `coast_dir` or `depth_d` cannot be parsed as numbers;
- `depth_d <= 0`;
- the input file cannot be opened;
- `output.csv` cannot be created;
- the input file has no header;
- the header does not start with `datetime,swh,mwp,mwd`.

10.2 Empty input after header

If the input file has a valid header but no data rows, the program:

- creates `output.csv` with only the header;
- creates `report.txt` stating that the input file contained no valid data;
- exits without treating the condition as a processing error.

10.3 Invalid or non-physical records

If a row cannot be parsed, or if:

$$swh \leq 0$$

or:

$$mwp \leq 0,$$

the derived output variables are written as zero.

10.4 Land-side waves

If:

$$0 < \text{relativeDir} < 180,$$

the record is classified as land-side and all locally transformed quantities are set to zero.

10.5 Small denominators

The code avoids unstable evaluations in shoaling and refraction by checking for near-zero denominators and using fallback values where needed.

10.6 Angle wrapping

The implementation uses two angular ranges:

- mean wave directions are wrapped to `[0, 360)`;
- signed obliquities are wrapped to `[-180, 180)`.

---

11. Practical interpretation

This executable is a point-transformation model. It is appropriate when the objective is to transform a long offshore time series to a fixed local depth using a transparent, reproducible, and computationally light method.

It should not be interpreted as a full coastal wave-propagation model. It does not represent:

- spatial bathymetric ray tracing;
- diffraction around structures or headlands;
- wave-current interaction;
- directional spreading;
- spectral partitioning;
- bottom-friction dissipation;
- surf-zone dissipation beyond the simple breaking cap;
- harbour resonance or basin penetration.

For design-critical applications, results should be checked against engineering judgement, local bathymetric context, and, where appropriate, a spectral wave model or site-specific wave-transformation study.

---

12. Build and compilation

The source is standard C++17 and uses OpenMP for parallel processing.

A typical compilation command is:

```
g++ -O3 -fopenmp -march=native -std=c++17 -Wall -Wextra -pedantic -Wconversion
 ↪  -Wsign-conversion -static -static-libgcc -static-libstdc++ -o transpose
 ↪  transpose.cpp -lm
```

12.1 Meaning of the main flags

- `-O3`: enables high compiler optimisation.
- `-fopenmp`: enables OpenMP multithreading.
- `-march=native`: tunes the executable for the compiling machine.
- `-std=c++17`: compiles as C++17.
- `-Wall -Wextra -pedantic`: enables strict warning checks.
- `-Wconversion -Wsign-conversion`: reports implicit numeric-conversion warnings.
- `-static -static-libgcc -static-libstdc++`: requests static linkage where supported.
- `-lm`: links the mathematical library where required.

On Windows with MSYS2/UCRT64, the same command can be used from the UCRT64 shell if `g++` and OpenMP support are installed.

---

13. Worked computational sequence

For one valid sea-side offshore record, the implemented chain is:

$$(mwp, H_s, MWD) \to L_0 \to L \to kh \to \alpha_{\text{offshore}} \to \alpha_{\text{local}} \to K_s, K_r \to H_b \to H_{s,\text{local}}, MWD_{\text{local}}.$$

The compact mathematical form is:

$$L_0 = \frac{gT^2}{2\pi},$$

$$L = L_0 \tanh\left(\frac{2\pi d}{L}\right), \qquad k = \frac{2\pi}{L}, \qquad kh = kd,$$

$$\sin(\alpha_{\text{local}}) = \sin(\alpha_{\text{offshore}}) \tanh(kh),$$

$$K_s = \left[\tanh(kh)\left(1 + \frac{2kh}{\sinh(2kh)}\right)\right]^{-1/2}, \qquad K_r = \sqrt{\frac{\cos(\alpha_{\text{offshore}})}{\cos(\alpha_{\text{local}})}},$$

$$H_b = 0.142L \tanh(kh), \qquad H_{s,\text{local}} = \min(H_{s,\text{offshore}} K_s K_r, H_b).$$

In this sequence, `T` is the input `mwp`.

---

14. Repository summary

In operational terms, `transpose.cpp`:

- reads offshore wave forcing from CSV;
- requires the first four input columns to be `datetime,swh,mwp,mwd`;
- ignores any additional input columns;

- sorts the input records by timestamp;
- removes duplicate timestamps;
- transforms each valid sea-side record to a prescribed depth;
- suppresses land-side waves;
- applies refraction, shoaling, and breaking limitation;
- writes a complete transformed time series to `output.csv`;
- writes descriptive statistics and annual maxima to `report.txt`.

The program is therefore a compact engineering tool for transparent offshore-to-nearshore wave transposition based on mean wave period, significant wave height, mean wave direction, coastline azimuth, and target local depth.